

Specification of the Angelfish consensus algorithm at a high level of abstraction. We use a number of abstractions and simplifying assumptions in order to expose the high-level principles of the algorithm clearly.

EXTENDS *Integers, FiniteSets, Sequences*

CONSTANTS

N The set of all nodes
 R The set of rounds
 $IsQuorum(-)$ Whether a set is a quorum (*i.e.* cardinality at least $n-f$)
 $IsBlocking(-)$ Whether a set is a blocking set (*i.e.* cardinality at least $f+1$)
 $Leader(-)$ operator mapping each round to its leader

ASSUME $\exists n \in R : R = 1 \dots n$ useful rounds start at 1

See *BlockDag* definitions in *BlockDag.tla*:

INSTANCE *BlockDag* WITH $N \leftarrow N, R \leftarrow R, Leader \leftarrow Leader$

A vote by a node for a vertex or for no vertex ($\langle \rangle$):

$Vote \triangleq N \times R \times (V \cup \{\langle \rangle\})$

$Vertex(v) \triangleq v[3]$

Timeout messages:

$Timeout \triangleq N \times R$

Quorum of messages:

$IsQuorumMsgs(M) \triangleq IsQuorum(\{Node(m) : m \in M\})$

--algorithm *Angelfish*{

variables

$vs = \{Genesis\}$, the vertices of the *DAG*

$es = \{\}$; the strong edges of the *DAG* (we do not model weak edges)

$votes = \{\}$; vote messages

$timeouts = \{\}$; timeout messages

define {

$dag \triangleq \langle vs, es \rangle$

whether we have timeout certificates for all rounds from $Round(l)$ to $rnd-1$:

$ValidPreviousLeader(l, rnd) \triangleq$

$\wedge IsLeader(l)$

$\wedge \forall r \in (Round(l)+1) \dots (rnd-1) :$

$IsQuorumMsgs(\{to \in timeouts : Round(to) = r\})$

whether a *DAG* d is valid:

$IsValid(d) \triangleq \forall v \in Vertices(d) : v \neq \langle \rangle \Rightarrow$

$\wedge Node(v) \in N \wedge Round(v) \in Nat \setminus \{0\}$

$\wedge \text{IF } \neg IsLeader(v)$

THEN $\forall v2 \in Children(d, v) : Round(v2) = Round(v) - 1$

```

ELSE  $\exists l \in \text{Children}(d, v) :$ 
     $\wedge \text{IsLeader}(l) \wedge \text{Round}(l) < \text{Round}(v)$ 
     $\wedge \text{ValidPreviousLeader}(l, \text{Round}(v))$ 
     $\wedge \forall v2 \in \text{Children}(d, v) :$ 
         $v2 \neq l \Rightarrow \text{Round}(v2) = \text{Round}(v) - 1$ 
}
macro StartRound1( ) {
    the first round is a bit special since there's no previous leader
    round := 1 ;
    either { create a vertex:
        vs := vs  $\cup$  { $\langle \text{self}, 1 \rangle$ } ;
        es := es  $\cup$  { $\langle \langle \text{self}, 1 \rangle, \text{Genesis} \rangle$ } edge to genesis
    }
    or or send an empty vote:
        votes := votes  $\cup$  { $\langle \text{self}, 1, \langle \rangle \rangle$ }
    }
macro Vote( deliveredVertices ) {
    if ( LeaderVertex(round)  $\in$  deliveredVertices  $\wedge$   $\langle \text{self}, \text{round} \rangle \notin \text{timeouts}$  )
        votes := votes  $\cup$  { $\langle \text{self}, \text{round} + 1, \text{LeaderVertex}(\text{round}) \rangle$ }
    else
        votes := votes  $\cup$  { $\langle \text{self}, \text{round} + 1, \langle \rangle \rangle$ }
    }
macro CreateVertex( receivedVertices ) {
    if ( LeaderVertex(round)  $\in$  receivedVertices  $\wedge$   $\langle \text{self}, \text{round} \rangle \notin \text{timeouts}$  ) {
        create a vertex pointing to the leader of the previous round
        with ( newV =  $\langle \text{self}, \text{round} + 1 \rangle$  ) {
            vs := vs  $\cup$  {newV} ;
            es := es  $\cup$  { $\langle \text{newV}, pv \rangle : pv \in \text{receivedVertices}$ }
        }
    }
    else {
        if ( Leader(round + 1)  $\neq$  self )
            create a vertex not pointing to any leader
            with ( newV =  $\langle \text{self}, \text{round} + 1 \rangle$  ) {
                vs := vs  $\cup$  {newV} ;
                es := es  $\cup$  { $\langle \text{newV}, pv \rangle : pv \in \text{deliveredVertices} \setminus \{ \text{LeaderVertex}(\text{round}) \}$ }
            }
        }
    }
    else
        we need to find a previous leader to point to
        with ( prevLeader  $\in$  { v  $\in$  vs :
            Round(v) < round  $\wedge$  v = LeaderVertex(Round(v)) } )
        with ( newV =  $\langle \text{self}, \text{round} + 1 \rangle$  ) {
            we must have timeout certificates down to the previous leader:
            when ValidPreviousLeader(prevLeader, round + 1) ;
            vs := vs  $\cup$  {newV} ;
        }
    }
}

```

```

        es := es
              ∪ {⟨newV, pv⟩ : pv ∈ deliveredVertices \ {LeaderVertex(round)}}
              ∪ {⟨newV, prevLeader⟩};   point to the previous leader
    }
  }
}
macro TryCommitLeader( deliveredVertices, deliveredVotes ) {
  with ( l = LeaderVertex(round - 1) )
  with ( support = vertices pointing to l plus votes
        {v ∈ deliveredVertices : ⟨v, l⟩ ∈ es} ∪ {vt ∈ deliveredVotes : vt[3] = l} )
  if ( IsQuorumMsgs(support) )
    log := Linearize(SubDag(dag, {l}), l);
  }
macro Catchup( ) {
  catch up to a round r strictly greater than round + 1
  with ( M ∈ SUBSET ((vs ∪ votes ∪ timeouts) \ {Genesis}) \ {[]} )
  with ( r = Min({Round(m) : m ∈ M}) )
  with ( B = {Node(m) : m ∈ M} ) {
    when r > round + 1 ∧ IsBlocking(B);
    round := r;
  }
}

```

Finally, we give the full specification of a node:

```

process ( correctNode ∈ N )
  variables
    round = 0,   current round; 0 means the node has not started execution
    log = ⟨⟩;   delivered log
  {
l0:   StartRound1();
l1:   while ( TRUE )
      either
        start the next round and create a vertex or vote
        with ( deliveredVertices ∈ SUBSET {v ∈ vs \ {⟨⟩} : Round(v) = round} )
        with ( deliveredVotes ∈ SUBSET {vt ∈ votes \ {⟨⟩} : Round(vt) = round} )
        with ( deliveredTimeouts ∈ SUBSET {to ∈ timeouts \ {⟨⟩} : Round(to) = round} ) {
          we heard from a quorum:
          when IsQuorumMsgs(deliveredVertices ∪ deliveredVotes ∪ deliveredTimeouts);
          we have the leader or a timeout certificate:
          when LeaderVertex(round) ∈ deliveredVertices ∨ IsQuorumMsgs(deliveredTimeouts);
          either {
            when (Leader(round + 1) ≠ self);   leaders cannot vote
              Vote(deliveredVertices)
          }
        }
      or

```

```

        CreateVertex(deliveredVertices);
        possibly commit the leader of round - 1:
    if ( round > 1 )
        TryCommitLeader(deliveredVertices, deliveredVotes);
        finally, increment the round counter:
        round := round + 1
    }
or
    timeout (this means we cannot support the leader of this round in the next round)
    timeouts := timeouts ∪ {⟨self, round⟩}
or
    Catchup()
}
}

```

Correctness properties:

$Compatible(s1, s2) \triangleq$ whether the sequence $s1$ is a prefix of the sequence $s2$, or vice versa
 $\forall i \in 1 \dots \text{Min}(\{Len(s1), Len(s2)\}) : s1[i] = s2[i]$

$Agreement \triangleq \forall n1, n2 \in N : Compatible(log[n1], log[n2])$

Basic typing invariant:

$TypeOK \triangleq$
 $\wedge IsValid(dag)$
 $\wedge \forall vt \in votes :$
 $\wedge vt = \langle vt[1], vt[2], vt[3] \rangle$
 $\wedge vt[1] \in N$
 $\wedge vt[2] \in Nat \setminus \{0\}$
 $\wedge vt[3] \in vs \cup \{ \langle \rangle \}$
 $\wedge \forall to \in timeouts :$
 $\wedge to[1] \in N$
 $\wedge to[2] \in Nat \setminus \{0\}$
 $\wedge \forall n \in N : round[n] \in Nat$